SMARTPICK AI

Product Requirements Document

PRD • Product vision, scope, user requirements and acceptance criteria

SmartPick AI — Intelligent Mobile Recommender

Prepared by Pratyush Mishra
Version 1.0 • 15 October 2026
Status: Draft for review

CONFIDENTIALITY NOTE

Prepared for academic and portfolio project planning. Verify dataset rights, technical assumptions, and implementation results before external publication.

Document Control

Field	Details
Document	Product Requirements Document (PRD)
Project	SmartPick AI — Intelligent Mobile Recommender
Version / Status	1.0 / Draft for review
Prepared by	Pratyush Mishra
Date	15 October 2026
Target delivery	3–7 days for a prototype; further iterations for production readiness

1. Executive Summary

SmartPick AI is a web-based smartphone recommendation application that helps users shortlist phones according to budget, brand preferences, and intended use such as gaming, photography, battery life, and everyday performance. The product will combine transparent filtering, a weighted preference-ranking baseline, and a content-based similarity method. Results will show why each phone was recommended rather than presenting a score as an objective measure of overall quality.

The first release is a portfolio-grade prototype built with Python and Streamlit. It is not a live retailer, price-comparison guarantee, or validated purchasing-advice service. Data provenance, freshness, and model limitations must be visible to users.

2. Problem Statement

Choosing a smartphone requires comparing specifications across multiple models while balancing budget and personal priorities. Users may struggle to identify suitable options or understand why one device ranks above another.

SmartPick AI addresses this by applying user-selected constraints and preferences to a curated smartphone dataset, then presenting a ranked shortlist with clear matching reasons.

3. Product Goals and Success Criteria

- Deliver an end-to-end working prototype within 3–7 days, subject to dataset availability and developer time.
- Allow a user to set a budget and at least three preference dimensions.
- Return up to five eligible recommendations, or a clear empty-state message when none qualify.
- Explain recommendations using the input preferences and available device attributes.
- Provide a side-by-side comparison view for selected phones.
- Document dataset source, update date, known gaps, and recommendation limitations.
- Publish reproducible setup instructions and tests in a GitHub-ready repository.

Prototype acceptance targets (to be verified, not assumed): 100% of displayed recommendations satisfy the budget constraint; valid input combinations return results or an understandable empty state; the app starts using documented commands; and the core ranking function passes unit tests.

4. Target Users and Use Cases

Persona	Need	Primary use case
Budget-conscious buyer	Shortlist phones within a fixed spend	Set minimum/maximum price and compare eligible phones
Gaming-focused user	Prioritize performance and display	Increase gaming/performance preference weights
Camera-focused user	Compare camera-related attributes	Prioritize available camera features while seeing their limits
Battery-focused user	Find phones with larger battery capacity	Rank phones by battery-related features
Student / project evaluator	Understand the ML workflow	Inspect methodology, evaluation, and dataset documentation

5. Scope

In scope — first release

- CSV-based smartphone dataset ingestion and validation.
- Budget filtering and optional brand/5G filters when fields are present.
- Preference-weighted ranking using available numeric attributes.
- Content-based similarity using appropriately preprocessed features.
- Top-five recommendations with score and human-readable rationale.
- Side-by-side comparison and basic exploratory charts.
- Input validation, empty states, error messages, and automated tests.
- README, requirements file, data dictionary, and deployment notes.

Out of scope — first release

- Guaranteed live pricing, stock availability, retailer checkout, or affiliate links.
- Medical, financial, or other high-stakes recommendations.
- Production-grade accounts, payments, or personal data collection.
- Claims of real-world predictive accuracy without representative labelled data.
- Collaborative filtering trained on genuine user interactions unless such data is lawfully available.
- Automated scraping that violates source terms or licensing.

6. Functional Requirements

ID	Requirement	Priority	Acceptance condition
FR-01	Load and validate the phone dataset	Must	Missing required columns or invalid types trigger a clear error.
FR-02	Set budget range	Must	Displayed results respect the configured budget rules.
FR-03	Set preference weights	Must	User can prioritize at least gaming/performance, camera, and battery.
FR-04	Generate ranked recommendations	Must	Return up to five eligible devices in descending score order.
FR-05	Explain each recommendation	Must	Explanation refers only to available attributes and user preferences.
FR-06	Compare selected phones	Should	Show available specifications side by side.
FR-07	Filter by brand / 5G	Should	Filters work when corresponding dataset columns exist.
FR-08	Show data caveats	Must	Display dataset source/as-of date and missing-data limitations.
FR-09	Show charts	Could	Provide basic price/specification distribution charts.
FR-10	Collect user feedback	Could	Only add after privacy and storage choices are defined.

7. User Journey

1. User opens the app and sees a short description plus dataset freshness note.
2. User selects budget, preferred brands (optional), and use-case priorities.
3. App validates inputs and filters ineligible phones.
4. Recommendation engine scores eligible devices and sorts them.

5. User reviews the Top 5, explanations, and available specifications.
6. User compares two or more phones or changes preferences to explore alternatives.

8. Product Experience and UI Requirements

- Use a clear title, concise instructions, and accessible labels for all controls.
- Group controls into Budget, Preferences, and Optional Filters.
- Display phone name, price, key attributes, match score, and reason for recommendation.
- Do not imply that a score is a probability or independent quality rating.
- Show an empty state if no phone meets the budget or filters.
- Use readable charts and responsive Streamlit layout; avoid requiring colour alone to communicate meaning.
- Keep data source and last-verified date visible.

9. Data Requirements

Proposed fields include phone_name, brand, price_inr, ram_gb, storage_gb, battery_mah, camera_mp, display_hz, has_5g, and processor_score where a trustworthy comparable source exists. Fields may be optional; the application must not silently invent missing values.

- Record source URL/name, retrieval date, licence/usage terms, and units for each dataset.
- Normalize units and categorical values; detect duplicate models and impossible values.
- Treat missing values explicitly and document any imputation.
- Do not treat megapixels as a complete camera-quality measure or battery capacity as a direct battery-life measurement.
- Only use ratings, benchmark scores, and prices when source and date are known.

10. Recommendation and Evaluation Requirements

Baseline: hard-filter by budget, then rank using normalized available features and user-specified non-negative weights. Advanced iteration: combine structured-feature similarity with text features such as brand/model/processor descriptions, where valid data is available.

- Fit preprocessing on the appropriate dataset split and reuse the fitted transformations consistently.
- Compare the advanced ranker with a simple baseline.
- Use Precision@K, Recall@K, or NDCG@K only when relevant-item labels or a defensible relevance-judgement set exists.
- Use synthetic labels only for functional tests; clearly label them synthetic and do not claim they prove real-world recommendation quality.
- Report data coverage, budget-constraint compliance, test results, and limitations.
- Use deterministic seeds and document evaluation setup for reproducibility.

11. Non-Functional Requirements

Area	Requirement
Usability	A first-time user should be able to obtain a recommendation without reading technical documentation.
Performance	For a small local CSV dataset (up to a few thousand rows), common interactions should feel responsive; benchmark on target hardware.
Reliability	Invalid input, missing file, empty results, and absent optional columns are handled gracefully.
Maintainability	Separate UI, loading/validation, recommendation, and evaluation modules.
Reproducibility	Pin or document dependencies; include setup and run instructions.
Privacy	Do not collect or store personal data by default.
Security	Do not commit secrets or private user data; validate uploaded files if upload functionality is later added.
Accessibility	Use descriptive labels, readable text, and clear error/empty states.

12. Risks and Mitigations

Risk	Mitigation
Incorrect/outdated phone data	Record sources and verification dates; show caveats and refresh data manually.
Biased or arbitrary feature weights	Expose weights, explain ranking, compare with baseline, and document design choices.
No relevance labels	Avoid unsupported accuracy claims; create a documented human-reviewed test set if feasible.
Limited build time	Prioritize budget filtering, ranking, Streamlit UI, and tests before optional features.
Dataset licence restrictions	Verify licence and attribution requirements before publishing data.
Deployment failures	Keep a local demo and screenshots as fallback; test dependency compatibility.

13. Release Plan

Milestone	Deliverable	Target
M1	Dataset schema, source notes, clean CSV	Day 1
M2	Baseline ranking and unit tests	Day 2
M3	Working Streamlit prototype	Day 3
M4	Similarity features and evaluation	Days 4–5
M5	README, repository, deployment attempt	Day 6
M6	Slides, demo script, viva preparation	Day 7

14. Acceptance Checklist

- App launches using the README instructions.
- Budget and preference controls affect results.
- No displayed result violates the budget filter.
- Top results are sorted consistently by the documented score.
- Explanation text does not claim unsupported facts.
- Empty dataset / no eligible phones / missing required columns are handled.
- Core logic has automated tests.
- Dataset source, licence, and as-of date are documented.
- Evaluation reports only metrics supported by available labels.

- Presentation distinguishes implemented features from future scope.

15. Future Enhancements

- User feedback and preference learning with explicit consent.
- Fresh price data from permitted APIs or licensed sources.
- More robust processor/GPU, camera, display, software-update, and repairability attributes.
- Hybrid recommendation using genuine interaction data if it becomes available.
- Model monitoring, data-version tracking, and periodic evaluation.
- Deployment hardening, accessibility improvements, and multilingual UI.

Appendix A — Assumptions

- The initial application is a student portfolio prototype.
- The developer will supply or select a lawful dataset; this document does not assert that a dataset has already been acquired.
- The project is expected to run locally first, with cloud deployment as a best-effort milestone.
- Actual delivery depends on the dataset, environment, and time available.